

How-To Guide

Building and Comparing Five Classical Search Algorithms on Cost-Weighted Terrain

CMPT 310 - D100, Introduction to Artificial Intelligence, Summer 2026 | Team Pathfinders: Banveet Johal, Lucas Zhang, Sheng Zhan

This guide walks through the non-trivial technical work behind our pathfinding project: turning a plain 2D grid into a **cost-weighted graph**, implementing five search algorithms from scratch, and building an evaluation/visualization pipeline to compare them fairly. It assumes you already know basic graph search (BFS/DFS) at the level covered in CMPT 310, and focuses on the parts that are easy to get subtly wrong: encoding movement cost, keeping algorithms comparable, and interpreting the results.

Step 1 - Represent Terrain as a Cost Surface, Not Just a Grid

The key design decision that makes this project non-trivial is that terrain has **elevation**, and moving between cells of different elevation costs more than moving between flat ones. We represent a scenario as a NumPy 2D array of elevation values plus a boolean obstacle mask, both of shape (rows, cols):

```
def generate_random_terrain(rows=25, cols=25, seed=42):
    rng = np.random.default_rng(seed)
    terrain = rng.normal(3, 1.5, size=(rows, cols))
    terrain = np.maximum(0, terrain)
    # smooth hill + scattered obstacles
    obstacles = rng.random((rows, cols)) < 0.12
    return {"terrain": terrain, "obstacles": obstacles, ...}
```

We built three scenarios this way (terrain.py): a flat baseline with uniform cost, a mountain barrier with a steep elevation ridge across the map, and a random terrain with scattered obstacles plus a smooth hill. Testing on all three - not just one - is what exposes the real differences between the algorithms later.

Step 2 - Turn the Grid into a Weighted Graph

Search algorithms don't operate on grids directly - they operate on graphs. We convert the grid into an adjacency list where each free cell is a node connected to its 4 neighbours, and the edge weight is **1 + the absolute elevation difference** between the two cells. This single formula is what makes UCS/A* meaningfully different from BFS/DFS: a 1-step move is never free, but a step onto much higher or lower ground costs more.

```
def movement_cost(current_elevation, next_elevation):
    return 1 + abs(next_elevation - current_elevation)

def build_graph(terrain, obstacles):
    graph = {}
    for row in range(rows):
        for col in range(cols):
            if obstacles[row, col]:
                continue # obstacles are not nodes at all
            for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                nr, nc = row+dr, col+dc
```

```

        if in_bounds(nr, nc) and not obstacles[nr, nc]:
            cost = movement_cost(terrain[row,col], terrain[nr,nc])
            graph[(row,col)].append(((nr,nc), cost))

    return graph

```

Tip: excluding obstacle cells from the graph entirely (rather than giving them infinite cost) keeps every algorithm's neighbour-expansion code identical - the only thing that differs between BFS, DFS, UCS, Greedy, and A* is how they choose which node to expand next.

Step 3 - Implement the Five Search Strategies

With a shared graph representation, each algorithm is just a different frontier data structure and expansion rule (`search.py`):

- **BFS** - a FIFO queue (`collections.deque`); explores in fewest-edges order, ignoring edge weights entirely.
- **DFS** - a stack; commits to one direction and backtracks, which tends to produce very long paths on a grid.
- **Uniform-Cost Search (UCS)** - a min-heap (`heapq`) keyed by accumulated path cost; always expands the cheapest known path first.
- **Greedy Best-First Search** - a min-heap keyed only by the Manhattan-distance heuristic to the goal; ignores cost-so-far completely.
- **A*** - a min-heap keyed by cost-so-far + heuristic, combining UCS's optimality guarantee with Greedy's goal-directed search.

```

def astar(graph, start, goal):
    frontier = [(manhattan_distance(start, goal), 0, start)]
    best_cost = {start: 0}
    while frontier:
        f, cost_so_far, current = heapq.heappop(frontier)
        if current == goal:
            break
        for neighbor, cost in graph[current]:
            new_cost = cost_so_far + cost
            if neighbor not in best_cost or new_cost < best_cost[neighbor]:
                best_cost[neighbor] = new_cost
                f_new = new_cost + manhattan_distance(neighbor, goal)
                heapq.heappush(frontier, (f_new, new_cost, neighbor))
    return reconstruct_path(parent, start, goal)

```

Tip: all five functions return the same shape of result (`{"algorithm", "path", "nodes_explored"}`), so the rest of the pipeline (metrics, plotting) never needs to know which algorithm produced a result.

Step 4 - Measure Every Run the Same Way

Nodes explored and runtime alone don't tell the whole story - an algorithm that explores very few nodes but finds an expensive path isn't necessarily "better". `metrics.py` walks the returned path and recomputes its true weighted cost and elevation gain against the graph, independent of what the search

algorithm itself optimized for:

```
def calculate_path_metrics(path, terrain, graph):
    total_cost = 0
    elevation_gain = 0
    for i in range(len(path) - 1):
        current, next_node = path[i], path[i + 1]
        if terrain[next_node] > terrain[current]:
            elevation_gain += terrain[next_node] - terrain[current]
        for neighbor, cost in graph[current]:
            if neighbor == next_node:
                total_cost += cost
    return {"weighted_path_cost": round(total_cost, 2), ...}
```

We also wrap every algorithm call with `time.perf_counter()` to record wall-clock runtime, giving four independent metrics per run: path length, weighted path cost, nodes explored, and runtime.

Step 5 - Visualize and Compare the Results

`visualize.py` draws two figures per scenario using Matplotlib: a side-by-side terrain map with each algorithm's explored path overlaid, and a bar-chart comparison of cost / nodes explored / runtime across all five algorithms. Both are saved automatically to `outputs/<scenario>_start.../` by `main.py`.

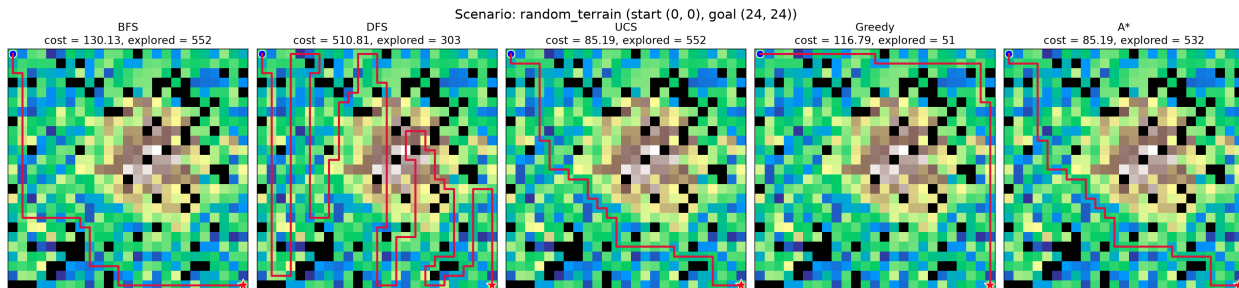


Figure 1. Explored path for each algorithm on the random-terrain scenario (start top-left, goal bottom-right). Notice how DFS zig-zags while UCS/A* trace a smooth, low-cost route.

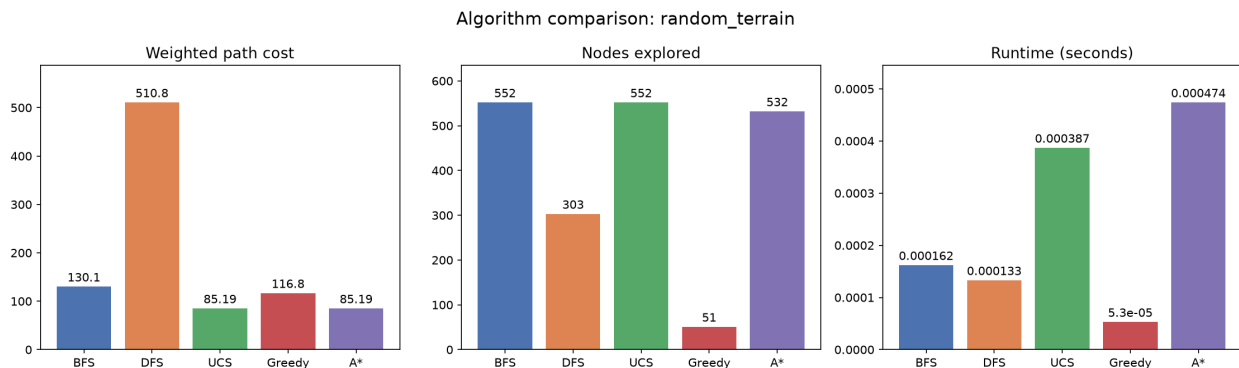


Figure 2. Weighted path cost, nodes explored, and runtime for all five algorithms on the same scenario. UCS and A* tie on optimal cost (85.19); Greedy explores far fewer nodes (51) but at a higher cost (116.79).

Troubleshooting Tips

Problem	Cause & Fix
DFS "finds a path" but the cost is far higher than every other algorithm.	This is expected, not a bug. DFS explores in a fixed neighbour order and commits to the first branch, without any notion of cost. If you expected DFS to find a short path, remember it only guarantees a path, not the cheapest or shortest one - compare <code>path_length</code> and <code>weighted_path_cost</code> separately to see this clearly.
KeyError or empty path when goal is unreachable.	<code>reconstruct_path()</code> returns <code>[]</code> when the goal was never added to the parent map. Always check <code>result["path_found"]</code> from <code>calculate_path_metrics</code> before indexing into the path - this can happen if obstacles fully wall off the goal.
Matplotlib hangs or errors with "no display" when running from a script/CI.	Set the non-interactive Agg backend before importing <code>pyplot</code> : <code>matplotlib.use("Agg")</code> (already done in <code>visualize.py</code>). This is required on headless machines and CI runners that have no GUI.

References

- Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson - BFS, DFS, Uniform-Cost Search, Greedy Best-First Search, and A* Search foundations.
- Python `heapq` and `collections` standard library documentation - used for the priority-queue and FIFO/stack frontiers.
- NumPy and Matplotlib official documentation - used for terrain arrays and all figures in this guide.
- Project repository: github.com/jameszhan02/CMPT310_Path_Finding ([cmpt310_milestone1_pathfinders/terrain_navigation/](https://github.com/jameszhan02/CMPT310_Path_Finding/tree/main/cmpt310_milestone1_pathfinders/terrain_navigation/)).